

Entregável C — Processo de Escoragem do Modelo

Documento para defesa final — Hackathon PodAcademy 2025

1. Visão Geral

O processo de escoragem (scoring) é a aplicação do modelo LightGBM treinado sobre novos dados para gerar **scores de risco de inadimplência (FPD)** para cada cliente. O score é uma probabilidade entre 0 e 1 — quanto maior, maior o risco de First Payment Default.

Resultado: KS OOT = **34.39%** (+1.29 p.p. acima do benchmark de 33.10%)

2. Artefatos do Modelo

O modelo gera e consome três tipos de artefatos, armazenados no bucket OCI

hackathon-2025-models :

```
hackathon-2025-models/
|
├─ pkl/
|   └─ modelo_fpd.pkl                # Modelo LightGBM serializado (pickle)
|
├─ resultados_modelo/
|   └─ predicoes_oot_{timestamp}.parquet    # Predições sobre dados OOT
|
└─ metricas/
    ├─ metricas_{timestamp}.json          # Métricas de performance
    └─ features_{timestamp}.txt            # Lista de features utilizadas
```

Artefato	Formato	Conteúdo
modelo_fpd.pkl	Python pickle	Modelo LightGBM Booster (~5-15 MB), ~900 iterações, 261 features
predicoes_oot_{ts}.parquet	Apache Parquet	~205K registros OOT com score_fpd e decil de risco
metricas_{ts}.json	JSON	KS, AUC, GINI (train + OOT), benchmark, gap, top features
features_{ts}.txt	Texto	Lista das 261 features selecionadas (IV >= 0.01)

3. Fluxo de Escoragem — 8 Etapas

O script `modelo_qualificacao.py` executa na VM E5.Flex (2 OCPUs, 32 GB RAM) na subnet privada da OCI. A VM é ligada sob demanda pelo Airflow e desligada após a execução.

Etapa 1 — Autenticação via Instance Principal

A VM autentica automaticamente na OCI via **Instance Principal** (Dynamic Group), sem chaves estáticas ou senhas. O signer é obtido pelo SDK OCI:

```
VM OCI → InstancePrincipalsSecurityTokenSigner → ObjectStorageClient → Namespace
```

Segurança: Sem credenciais hardcoded. A autenticação é gerenciada pela IAM da OCI via Dynamic Group associado ao OCID da VM.

Etapa 2 — Leitura da ABT v6 (com filtros na leitura)

A ABT v6 é lida do bucket `hackathon-2025-gold-layer/abt_v6_v2/` via OCI SDK (pandas, sem Spark). Filtros aplicados **durante a leitura** de cada chunk:

- `flag_instalacao_int = 1` (apenas clientes com FPD observável)
- `safra IN ('202410', '202411', '202412', '202502', '202503')`

Otimização de memória:

- Downcast float64 → float32 e int64 → int32 em cada chunk
- Dedup incremental a cada 40 arquivos (evita OOM com re-execuções)
- Pico de memória: ~16 GB dos 32 GB disponíveis

Métrica	Valor
Arquivos parquet lidos	~160
Registros após filtro + dedup	~535K
Colunas	614
Memória	~16 GB

Etapa 3 — Split Temporal

Divisão por safra (não aleatória) para simular condições reais de produção:

Out/2024	Nov/2024	Dez/2024		Fev/2025	Mar/2025
TREINO (~330K CPFs)				OOT (~205K CPFs)	
Modelo aprende com esses				Modelo NUNCA viu esses	
				→ são o "teste real"	

Conjunto	Safras	Registros	Papel
Treino	202410, 202411, 202412	~330.056	Modelo aprende padrões
OOT	202502, 202503	~205.462	Validação em dados nunca vistos

Etapa 4 — Seleção de Features por Information Value (IV)

Para cada feature numérica (excluindo colunas meta como num_cpf, safra, fpd_int, flag_instalacao_int):

1. Calcula cobertura (% não-nulo) — descarta se < 1%
2. Calcula IV (Information Value) usando 10 bins (quantis)
3. Seleciona features com IV >= 0.01

Entrada	Saída
614 colunas na ABT v6	261 features selecionadas

Entrada	Saída
Colunas meta	Removidas (anti-leakage)
Colunas com IV < 0.01	Descartadas

Por que IV = 0.01 e não 0.02? Features comportamentais (Recarga, Pagamento, Atraso) têm IV individual baixo (0.01-0.04), mas contribuem **+5.13 p.p. de KS** coletivamente. Um limiar de 0.02 excluiria features valiosas.

Etapa 5 — Treino ou Carga do Modelo (train-or-load)

O script implementa lógica **train-or-load**:

- 1. Tenta carregar `pk1/modelo_fpd.pk1` do bucket
- 2. **Se encontra:** pula treino, vai direto para scoring (re-scoring mensal)
- 3. **Se não encontra:** treina novo modelo LightGBM

Hiperparâmetros do LightGBM:

Parâmetro	Valor	Justificativa
num_leaves	31	Complexidade moderada (evita overfitting)
max_depth	6	Limita profundidade das árvores
learning_rate	0.05	Taxa de aprendizado conservadora
feature_fraction	0.8	Usa 80% das features por árvore (regularização)
bagging_fraction	0.8	Usa 80% das amostras por iteração
min_child_samples	100	Mínimo de registros por folha
early_stopping	50 rounds	Para se AUC no OOT não melhorar em 50 iterações
num_boost_round	1000 (máx)	Melhor iteração encontrada: ~900

Etapa 6 — Predições e Métricas

O modelo gera probabilidades de FPD para os conjuntos de treino e OOT:

Métrica	Treino	OOT
KS	38.12%	34.39%
AUC	0.7654	0.7327
GINI	53.08%	46.54%
Benchmark	—	33.10%
Gap vs Benchmark	—	+1.29 p.p.

Interpretação: Gap treino-OOT de ~4 p.p. indica modelo sem overfitting significativo. O KS OOT supera o benchmark fornecido pela Claro.

Etapa 7 — Exportação dos Resultados

Três artefatos são salvos no bucket `hackathon-2025-models` via OCI SDK:

7a. Predições OOT (parquet):

Coluna	Tipo	Descrição
num_cpf	int32	Identificador do cliente
safra	string	Mês da operação (202502 ou 202503)
fpd_int	int32	Target real (0=bom, 1=mau) — para validação
score_fpd	float32	Probabilidade de FPD (0.0 a 1.0)
decil	int32	Decil de risco (1=menor risco, 10=maior risco)

7b. Métricas (JSON): KS, AUC, GINI, benchmark, gap, top 10 features, contagens treino/OOT.

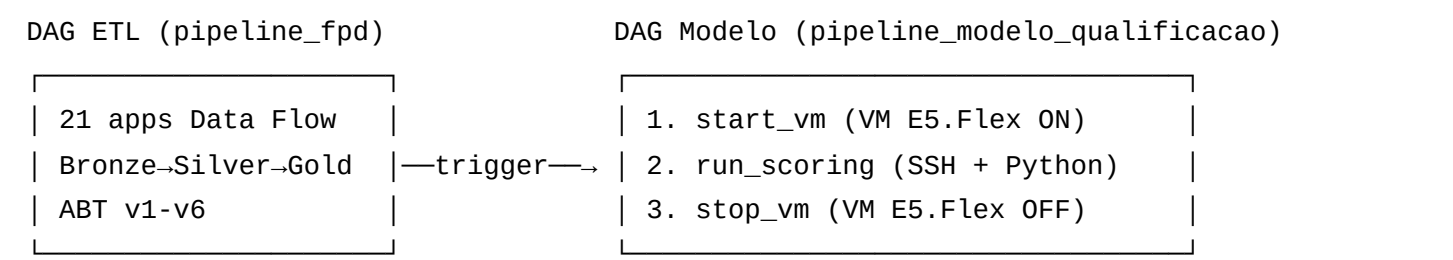
7c. Features (TXT): Lista completa das 261 features selecionadas, uma por linha.

Etapa 8 — Resumo e Finalização

O script imprime o resumo final com status (ACIMA/ABAIXO DO BENCHMARK) e paths dos artefatos salvos. A VM é desligada automaticamente pelo Airflow após a execução.

4. Orquestração via Airflow

O scoring é orquestrado pela DAG `pipeline_modelo_qualificacao` no Airflow:



Passo	Ação	Detalhe
<code>start_vm</code>	Liga VM E5.Flex	OCI SDK <code>compute_client.instance_action("START")</code>
<code>run_scoring</code>	Executa script via SSH	<code>python3.11 /opt/modelo-fpd/modelo_qualificacao.py</code>
<code>stop_vm</code>	Desliga VM	<code>trigger_rule=ALL_DONE</code> — desliga mesmo se scoring falhar

Encadeamento automático: A DAG ETL usa `TriggerDagRunOperator` para disparar a DAG do modelo automaticamente após o pipeline completo. A DAG modelo também pode ser executada manualmente (re-scoring sem re-ETL).

5. Ciclo de Vida do Modelo

Re-scoring Mensal (cenário padrão)

Novos dados → ETL (Bronze→ABT) → Carrega PKL existente → Score novos clientes → Salva pi

O modelo treinado com safras out/nov/dez 2024 é **reutilizado** para escorar novos clientes. Não precisa retreinar todo mês.

Quando Retreinar

Sinal	Critério	Ação
Queda de KS	KS OOT cai > 2 p.p.	Retreinar com safras recentes
Data drift	Distribuição de features muda significativamente	Investigar + retreinar
Mudança de política	Claro altera critérios de instalação	Retreinar com dados pós-mudança
Governança	A cada 6 meses (obrigatório)	Deletar PKL → pipeline treina automaticamente

Como Forçar Retreino

```
# Executar no Local/WSL:
oci os object delete -bn hackathon-2025-models --name pkl/modelo_fpd.pkl --force
# Próxima execução do pipeline treina modelo do zero
```

6. Segurança e Anti-Leakage

Proteções implementadas

Proteção	Implementação
Anti-leakage	fpd_int e flag_instalacao_int na lista META_COLS , excluídos das features
Split temporal	Treino com passado (out-dez/2024), teste com futuro (fev-mar/2025)
Sem chaves estáticas	Instance Principal via Dynamic Group (IAM OCI)
Subnet privada	VM sem IP público, acesso apenas via jump host (Airflow VM)
Audit trail	Cada run gera métricas com timestamp — histórico completo no bucket

Proteção	Implementação
Feature list auditável	features_{ts}.txt comprova que colunas proibidas não foram usadas

Verificação de anti-leakage

Para auditar que o modelo não usa colunas proibidas:

```
# Executar no Local/WSL:
oci os object get -bn hackathon-2025-models --name metricas/features_20260305_1721.txt .
# Resultado esperado: vazio (nenhuma correspondência)
```

7. Custos da Escoragem

Componente	Custo	Detalhe
VM E5.Flex (2 OCPUs, 32 GB)	~R\$0 quando parada	Start/Stop — ligada apenas durante scoring
Tempo de execução	~15 minutos	Leitura + scoring + exportação
Object Storage (artefatos)	< R\$1/mês	PKL (~10 MB) + parquets + JSONs
Total por execução	~R\$2-3	Compute-hours da VM durante 15 min

8. Reprodutibilidade

Todos os elementos necessários para reproduzir o scoring estão versionados:

Elemento	Localização
Script de scoring	mig_oci/data_science/scripts/modelo_qualificacao.py
Modelo PKL	hackathon-2025-models/pkl/modelo_fpd.pkl (OCI Object Storage)
Dados ABT v6	hackathon-2025-gold-layer/abt_v6_v2/ (OCI Object Storage)

Elemento	Localização
Infraestrutura	mig_oci/terraform/ (100% IaC)
Orquestração	mig_oci/airflow/dags/dag_modelo_qualificacao.py
Deploy	mig_oci/airflow/deploy_modelo.sh
Métricas históricas	hackathon-2025-models/metricas/ (JSON + TXT por run)

Para reproduzir na OCI

```
# 1. Deploy do script (Executar no Local):
cd mig_oci/airflow
./deploy_modelo.sh <AIRFLOW_IP> <MODELO_IP> ~/.ssh/airflow_vm ~/.ssh/modelo_vm

# 2. Trigger via Airflow (Executar na VM Airflow):
docker compose exec airflow-scheduler airflow dags trigger pipeline_modelo_qualificacao

# 3. Ou trigger automático após ETL (Executar na VM Airflow):
docker compose exec airflow-scheduler airflow dags trigger pipeline_fpd
```

9. Números-Chave

Métrica	Valor
Features na ABT v6	614
Features selecionadas (IV >= 0.01)	261
Registros de treino	330.056
Registros OOT	205.462
KS OOT	34.39%
Benchmark	33.10%
Gap vs benchmark	+1.29 p.p.
AUC OOT	0.7327

Métrica	Valor
GINI OOT	46.54%
Melhor iteração (early stopping)	~900
Tempo de execução	~15 minutos
Custo por execução	~R\$2-3